

Аналитическая записка к лабораторной работе

«Поиск лиц на фотографиях: каскад Хаара и нейросеть YUNet»

Могилев Антон, группа Э-ОМ-093-13-26-01 (ЭМИТ)

09.04.03 Прикладная информатика / маг ОПИ26к

1. Задача и данные

Программа на Python и OpenCV проходит по папке `faces/`, находит на фото лица, обводит их рамками и делит файлы на «с лицами» и «без лиц». Требования: находить и повернутые лица, не срабатывать на предметы и животных, работать на обычном процессоре, использовать готовую модель.

Таблица 1 — Тестовый набор

Файл	Что на фото	Лиц
beatles.jpg	Коллаж 2×2, голова Леннона (вверху слева) повернута вбок	4
messi.jpg, woman.jpg	Портреты анфас	по 1
fruits.jpg, owl.png, vase.png	Фрукты, сова, ваза — проверка ложных срабатываний	0

2. Как работают модели

Каскад Хаара — метод Виолы–Джонса (2001), модель `haarcascade_frontalface_default`. Изображение переводится в оттенки серого, по нему скользит окно 24×24 пикселя, а картинка постепенно уменьшается, чтобы найти лица разного размера. В каждом окне проверяются простые шаблоны светотени («глаза темнее щёк», «переносица светлее глаз»), выстроенные в цепочку от простых к сложным. Шаблоны обучены **только на лицах анфас**. Модель возвращает лишь рамки, без оценки уверенности.

YUNet (`face_detection_yunet_2023mar.onnx`, 2023) — маленькая свёрточная нейросеть (232 КБ, около 53 тыс. весов), встроенная в OpenCV как `FaceDetectorYN`. Признаки лица она не получает готовыми, а выучила сама на наборе WIDER FACE, где лица сняты под разными углами, частично закрыты и по-разному освещены. За один проход сеть оценивает кандидатов на трёх сетках (шаг 8, 16 и 32 пикселя — для мелких, средних и крупных лиц) и для каждого возвращает рамку, 5 ключевых точек (глаза, нос, уголки рта) и уверенность от 0 до 1. Затем слабые кандидаты отсекаются порогом, а дубли одного лица удаляются (NMS).

3. Параметры

Таблица 2 — Параметры каскада Хаара (detectMultiScale)

Параметр	Значение	На что влияет
scaleFactor	1.1	Шаг уменьшения картинки. Меньше — медленнее и больше ложных рамок (1,03: 4 «лица» на сове); больше — лица «между шагами» теряются (1,3: потеряно лицо Месси)
minNeighbors	5	Сколько соседних срабатываний нужно для лица. Меньше — выше чувствительность и много ложных рамок (при 1 — 12 ошибок); больше — больше пропусков
minSize / maxSize	не заданы	Границы размера лица; отсекают слишком мелкие или крупные находки

Таблица 3 — Параметры YuNet (FaceDetectorYN) в программе

Параметр	Значение	На что влияет
input_size	размер фото	Размер входа сети, задаётся для каждого фото через setInputSize. Слишком маленький — теряются мелкие лица (beatles.jpg при 120×120 — 4 лица, при 80×80 — 0)
score_threshold	0.8	Минимальная уверенность для лица — главный параметр. Ниже — больше находок и ложных рамок, выше — риск пропусков
nms_threshold	0.3	Насколько могут перекрываться рамки одного лица. При 0,3 на beatles.jpg — 4 рамки, при 0,9 — 6, при 1,0 (без удаления дублей) — 31
top_k	5000	Сколько кандидатов рассматривается до удаления дублей; на результат почти не влияет
backend_id / target_id	OpenCV / CPU	Чем и на чём считать: встроенный движок на процессоре

У каскада строгость регулируется косвенно двумя параметрами, которые по-разному действуют на разные фото. У YuNet за строгость отвечает один понятный параметр — порог уверенности.

4. Результаты

Параметры Хаара — стандартные (1.1 и 5), YuNet — из таблицы 3. Время — среднее из 5 запусков на процессоре (OpenCV 4.14).

Таблица 4 — Сравнение на тестовом наборе

Файл	Лиц	Хаар	YuNet	Хаар, мс	YuNet, мс
beatles.jpg	4	3	4	64	17
messi.jpg	1	1	1	20	6
woman.jpg	1	1	1	15	6
fruits.jpg	0	0	0	14	5
owl.png	0	0	0	39	33
vase.png	0	0	0	53	25
Итого	6	5 из 6	6 из 6	205	92

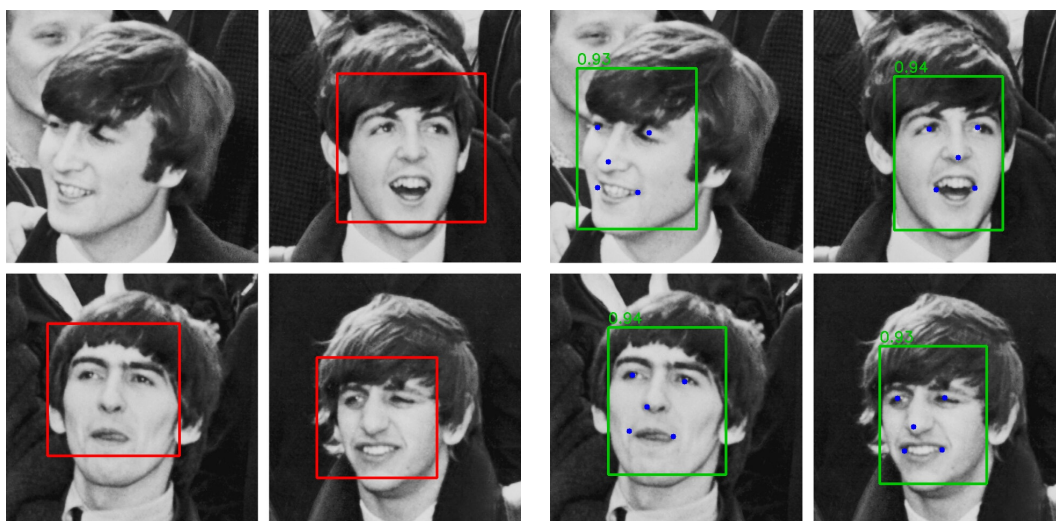


Рисунок 1 — beatles.jpg: слева каскад Хаара (лицо Леннона пропущено), справа YuNet (все 4 лица, уверенность 0,93–0,94)

Настройка Хаара не помогает. При `minNeighbors` от 1 до 8 и `scaleFactor` от 1,03 до 1,3 лицо Леннона не находится ни разу. Зато при ослаблении растут ложные рамки: при `minNeighbors` = 1 их 12, в том числе 5 на сове (рисунок 2), и все три фото без людей попали бы в группу «с лицами».

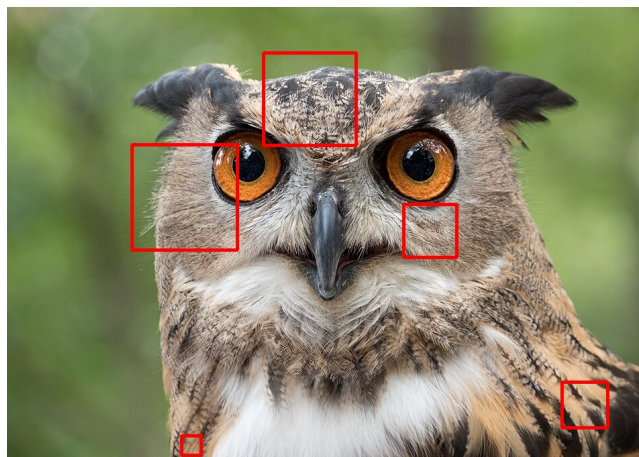


Рисунок 2 — Каскад Хаара при `minNeighbors` = 1: пять ложных «лиц» на сове

Другие каскады тоже не помогают. Варианты `frontalface_alt` и `alt2` находят те же 3 лица из 4, `alt_tree` — ни одного. Каскад профиля (`profileface`, на исходном и отражённом фото) находит трёх других участников группы, но не Леннона: его голова повернута на три четверти, а не строго в профиль.

Порог YuNet. Настоящие лица получили уверенность от 0,909, ложные кандидаты — не больше 0,514 (фон с трибунами на `messi.jpg`); на сове — до 0,2. Порог 0,8 лежит внутри этого разрыва с запасом.

5. Выводы

Почему `frontalface` не подходит:

- обучена только на лицах анфас и не находит повернутые лица (пропуск 1 из 6);
- это не исправить настройкой: ослабление параметров даёт до 12 ложных рамок, а пропуск остаётся;
- другие каскады анфаса и каскад профиля не находят промежуточный поворот головы;
- нет оценки уверенности, поэтому строгость подбирается вслепую.

Почему выбрана YuNet:

- нашла все 6 лиц, включая повернутое, и ни одного ложного;
- строгость задаётся одним порогом уверенности, выбранным по данным;
- в 2,2 раза быстрее каскада на обычном процессоре;
- встроена в OpenCV, весит 232 КБ, не требует видеокарты и других библиотек.

Источники

1. Viola P., Jones M. Rapid Object Detection using a Boosted Cascade of Simple Features // CVPR. — 2001.
2. Wu W., Peng H., Yu S. YuNet: A Tiny Millisecond-level Face Detector // Machine Intelligence Research. — 2023.
3. Yang S. et al. WIDER FACE: A Face Detection Benchmark // CVPR. — 2016.
4. OpenCV Zoo, YuNet. — URL: https://github.com/opencv/opencv_zoo

Приложение. Исходная версия на каскаде Хаара

Первая версия программы в репозиторий не сохранялась, ниже её реконструкция. Отличия от итоговой: CascadeClassifier вместо FaceDetectorYN, перевод в оттенки серого и detectMultiScale вместо detect. Файл модели входит в пакет opencv-python. Её результат на beatles.jpg — рисунок 1 (слева).

```
import cv2
import os

folder_dir = "./faces"
results_dir = "./results"

# классический детектор: каскад Хаара, обученный на лицах анфас
face_detector = cv2.CascadeClassifier(
    cv2.data.haarcascades + "haarcascade_frontalface_default.xml")

with_faces = []
without_faces = []

def main():
    os.makedirs(results_dir, exist_ok=True)

    for filename in os.listdir(folder_dir):
        if not filename.lower().endswith((".jpg", ".jpeg", ".png")):
            continue

        image = cv2.imread(os.path.join(folder_dir, filename))
        if image is None:
            print(f"error reading file: {filename}")
            continue

        # каскад работает только с изображением в оттенках серого
        gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
        faces = face_detector.detectMultiScale(
            gray,
            scaleFactor=1.1,    # шаг уменьшения изображения
            minNeighbors=5,      # сколько соседних срабатываний нужно для лица
        )

        if len(faces) == 0:
            without_faces.append(filename)
            continue

        with_faces.append(filename)
        for (x, y, w, h) in faces:
            cv2.rectangle(image, (x, y), (x + w, y + h), (0, 255, 0), 2)
            cv2.imwrite(os.path.join(results_dir, filename), image)

        for name in with_faces:
            print("+ ", name)
        for name in without_faces:
            print("- ", name)

if __name__ == '__main__':
    main()
```